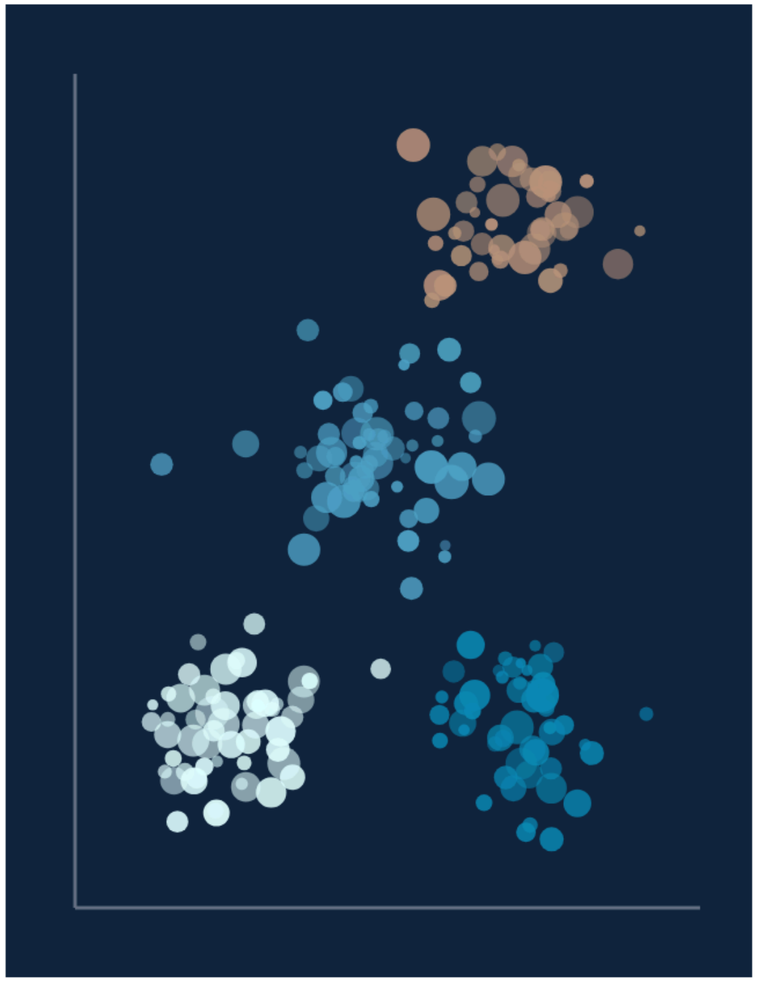
PROJET 13 · DATA SCIENCE

Segmentation des clients selon leur valeur financière



Modélisation RFM · K-means, CHA (Ward), GMM

Jeu de données UCI Online Retail (2010–2011)

Basma EN-NIYARI · Sanae EL RHARABY

FING2

Démarche : 12 étapes, chacune avec son code Python

Étapes 1–3

Charger, auditer et nettoyer

Étapes 4–6

Explorer, construire RFM, transformer

Étapes 7–9

Choisir K, modéliser, tester la stabilité

Étapes 10–12

Profiler, visualiser (ACP), valider dans le temps

Question : peut-on regrouper les clients en segments de valeur homogènes, stables et actionnables ?

1 Importer les bibliothèques et charger les données

Comment : pandas lit le fichier Excel monté depuis Google Drive ; scikit-learn fournit les algorithmes de clustering.

Code python

```
from google.colab import drive
drive.mount("/content/drive")

import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

DATA_PATH = "/content/drive/MyDrive/projet 13/
            Online Retail.xlsx"
raw = pd.read_excel(DATA_PATH)
print(raw.shape)       # (541909, 8)
print(raw.dtypes)
```

541 909
lignes brutes

8
variables

4 372
clients distincts (avec ID)

25 %
lignes sans CustomerID

Période : 01/12/2010 → 09/12/2011

2 Auditer la qualité des données

Comment : on compte, avec des masques booléens pandas, chaque anomalie avant de décider quoi supprimer.

Code python

```
audit = {}
audit["sans_client"] = raw.CustomerID.isna().sum()
est_c = raw.InvoiceNo.str.startswith("C")
audit["annulations"] = est_c.sum()
audit["prix<=0"] = (raw.UnitPrice <= 0).sum()
audit["doublons"] = raw.duplicated().sum()

CODES = ["POST", "D", "M", "DOT", "BANK CHARGES"]
audit["codes_frais"] = raw.StockCode.isin(CODES).sum()

print(pd.Series(audit))
```

Anomalie	Lignes
Sans CustomerID	135 080
Annulations (C…)	9 288
Prix ≤ 0	2 517
Doublons exacts	5 268
Codes de frais	2 771

3 Nettoyer et séparer ventes et annulations

Comment : on filtre pandas dans un ordre précis, et on garde les annulations à part pour calculer un taux par client.

Code python

```
df = raw.drop_duplicates()
df = df[df.CustomerID.notna()]
df["CustomerID"] = df.CustomerID.astype(int)
df = df[~df.StockCode.isin(CODES)]

df["Montant"] = df.Quantity * df.UnitPrice

est_annul = df.InvoiceNo.str.startswith("C")
annul = df[est_annul]
ventes = df[~est_annul
            & (df.Quantity > 0)
            & (df.UnitPrice > 0)]
print(ventes.CustomerID.nunique())    # 4334
```

541 909 brut

536 641 sans doublons

401 604 avec client

391 283 lignes de vente

+ 8 507 annulations conservées à part

4 Explorer : la concentration du chiffre d’affaires

Comment : on agrège le CA par client, on trie, puis on calcule la part cumulée (courbe de Lorenz / Pareto).

Code python

```
ca_client = (ventes.groupby("CustomerID")
             .Montant.sum()
             .sort_values(ascending=False))

cum = ca_client.cumsum() / ca_client.sum()
top20 = cum.iloc[int(0.2 * len(cum)) - 1]
print(f"Top 20 % des clients = {top20:.0%} du CA")

lorenz = np.sort(ca_client.values).cumsum()
lorenz = lorenz / ca_client.sum()
gini = 1 - 2 * np.trapezoid(lorenz, dx=1/len(lorenz))

plt.plot(cum.values)    # courbe de concentration
```

75 %	du CA : top 20 % des clients
32 %	du CA : top 1 %
0,72	indice de Gini
83 %	du CA au Royaume-Uni

+ 8 507 annulations conservées à part

5 Construire le profil financier RFM par client

Comment : un groupby sur CustomerID calcule récence, fréquence et montant à une date de référence fixe.

Code python

```
def construire_features(v, a, snapshot):
    g = v.groupby("CustomerID")
    f = pd.DataFrame({
      "Recency": (snapshot - g.InvoiceDate.max()).dt.days,
      "Frequency": g.InvoiceNo.nunique(),
      "Monetary": g.Montant.sum()})
    # + AvgBasket, ReturnRate, NbProducts…
    return f

SNAPSHOT = (ventes.InvoiceDate.max().normalize()
            + pd.Timedelta(days=1))    # 10/12/2011
feat = construire_features(ventes, annul, SNAPSHOT)
print(feat.describe())
```

Variable	Définition	Médiane
Recency	jours depuis le dernier achat	50 j
Frequency	nb de factures	2
Monetary	CA cumulé	664 £

6 Transformer et standardiser les variables

Comment : log(1+x) corrige l'asymétrie, StandardScaler met les variables à la même échelle (moyenne 0, écart-type 1).

Code python

```
core = ["Recency", "Frequency", "Monetary"]

print(feat.Monetary.skew())              # 19.6
X = np.log1p(feat[core])                 # log(1 + x)
print(X.Monetary.skew())                 # 0.4

scaler = StandardScaler().fit(X)
X = scaler.transform(X)

# X est la matrice 4334 x 3 utilisée pour tous les
# algorithmes de clustering (basés sur les distances)
```

Asymétrie	Brute	Après log
Recency	1,2	-0,5
Frequency	12,0	1,2
Monetary	19,6	0,4

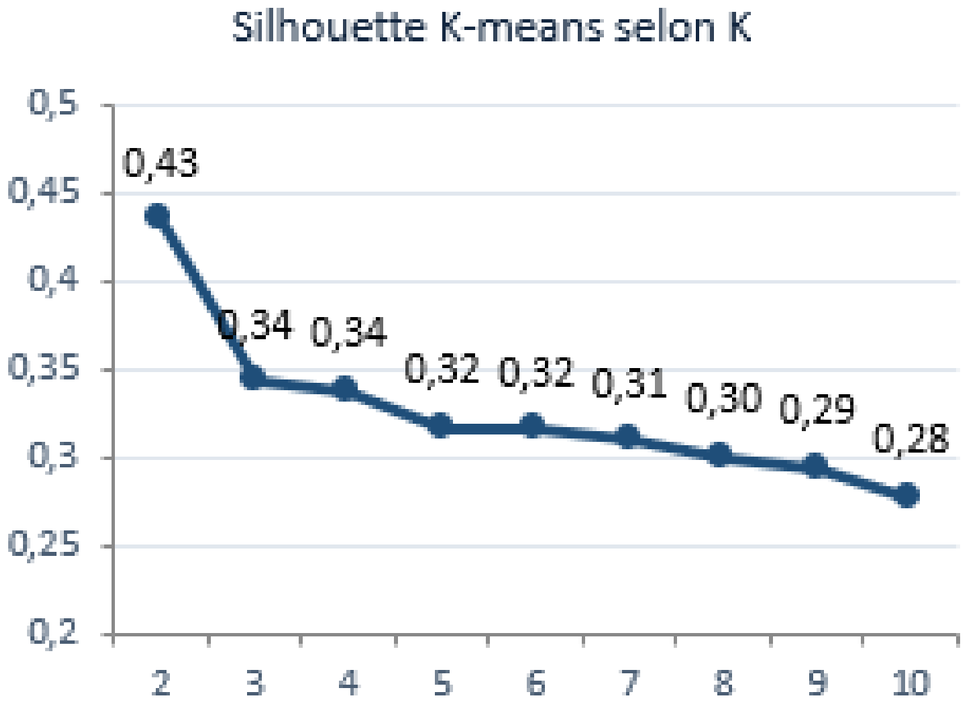
7 Choisir le nombre de segments K

Comment : on ajuste les modèles pour K = 2 à 10 et on compare silhouette, inertie, Davies-Bouldin et BIC.

Code python

```
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage

Z = linkage(X, method="ward")
for k in range(2, 11):
    km = KMeans(n_clusters=k, n_init=20,
                random_state=42).fit(X)
    gm = GaussianMixture(k, covariance_type="full",
            n_init=5, reg_covar=1e-2,
            random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    bic = gm.bic(X)
    inertie = km.inertia_    # méthode du coude
```



K = 4 retenu : bon compromis entre score, stabilité et lisibilité.

8 Ajuster K-means, CHA (Ward) et GMM

Comment : trois modèles sont ajustés avec K = 4 sur la même matrice X, puis comparés aux mêmes critères.

Code python

```
from scipy.cluster.hierarchy import fcluster
K = 4

km = KMeans(n_clusters=K, n_init=50,
            random_state=42).fit(X)

Z = linkage(X, method="ward")
lab_cha = fcluster(Z, K, criterion="maxclust") - 1

gm = GaussianMixture(K, covariance_type="full",
        n_init=10, reg_covar=1e-2,
        random_state=42).fit(X)
lab_gmm = gm.predict(X)

for lab in (km.labels_, lab_cha, lab_gmm):
    print(silhouette_score(X, lab),
          davies_bouldin_score(X, lab))
```

K = 4	K-m.	CHA	GMM
Silhouette	0,34	0,28	0,19
Calinski-H.	3 300	2 774	2 270
Davies-B.	1,01	1,11	1,58
Stabilité	0,96	0,39	0,88

K-means retenu : meilleurs scores et meilleure stabilité.

9 Mesurer la stabilité des segments

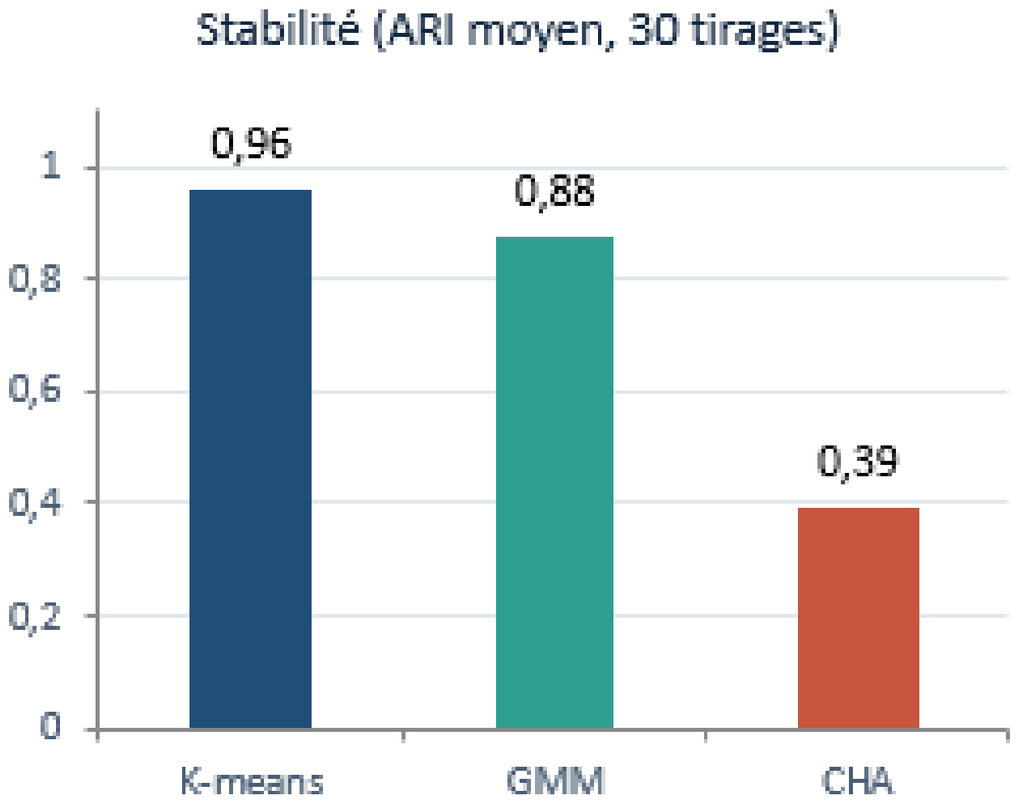
Comment : on refait le clustering sur 80 % des clients tirés au hasard, 30 fois, et on mesure l'accord (ARI) avec la partition de référence.

Code python

```
from sklearn.metrics import adjusted_rand_score
rng = np.random.default_rng(42)
n = len(X); ari = []

for b in range(30):
    idx = rng.choice(n, size=int(0.8 * n),
                     replace=False)
    new = KMeans(4, n_init=10,
                 random_state=b).fit_predict(X[idx])
    ari.append(adjusted_rand_score(
        km.labels_[idx], new))

print(np.mean(ari), np.std(ari))    # 0.96, 0.04
```



10 Profiler et nommer les segments

Comment : groupby sur le segment, statistiques médianes par variable, puis règles lisibles pour attribuer un nom.

Code python

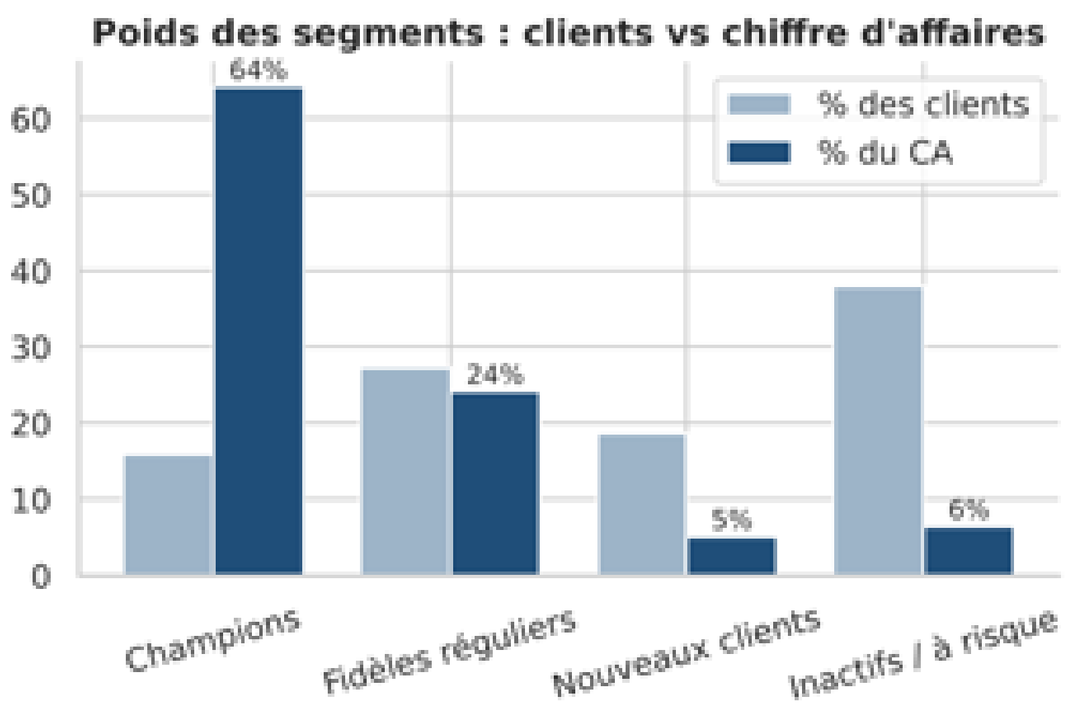
```
feat["Segment_id"] = reordonner(km.labels_, feat)

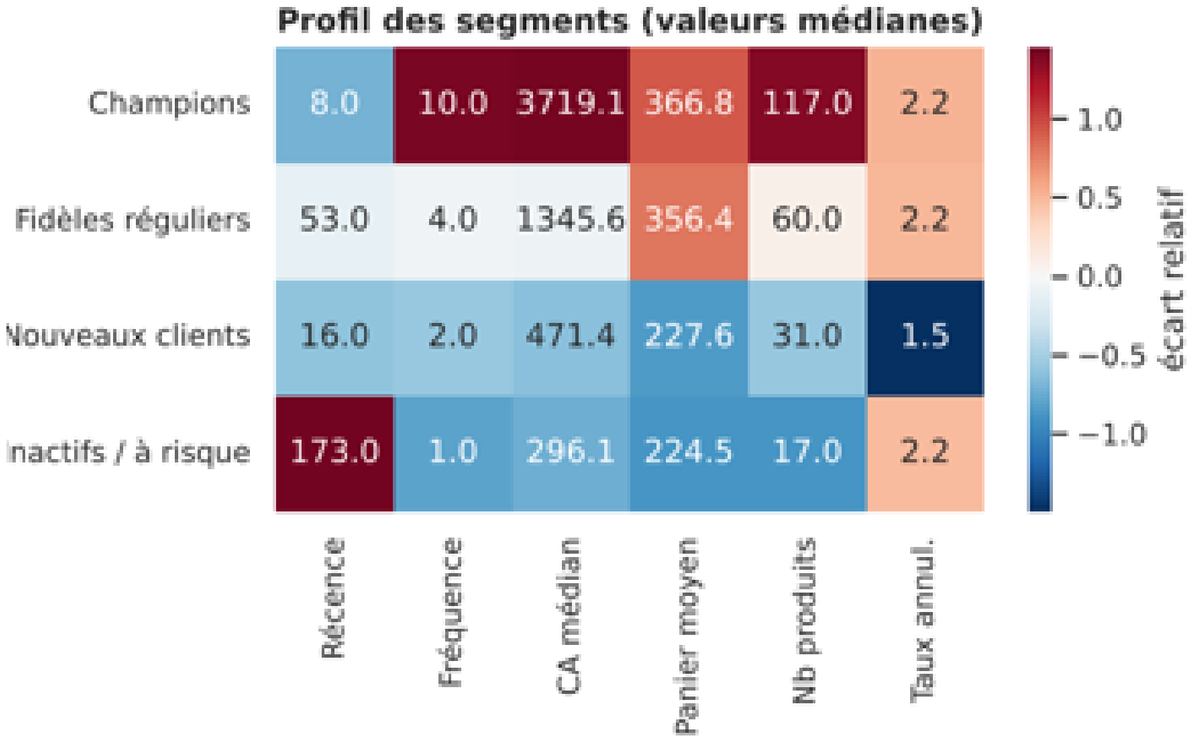
prof = feat.groupby("Segment_id").agg(
    Clients=("Monetary", "size"),
    Recency_med=("Recency", "median"),
    Frequency_med=("Frequency", "median"),
    Monetary_med=("Monetary", "median"),
    CA_total=("Monetary", "sum"))
prof["Part_CA_%"] = (100 * prof.CA_total
                     / prof.CA_total.sum())

# Nommage : CA médian max = Champions,
# récence max = Inactifs, etc.
feat["Segment"] = feat.Segment_id.map(NOMS)
```

Segment	Clients	% CA	R (j)	F
Champions	688	64 %	8	10
Fidèles	1 183	24 %	53	4
Nouveaux	813	5 %	16	2
Inactifs	1 650	6 %	173	1

Les Champions : 16 % des clients, 64 % du chiffre d’affaires





Résultat du code de l’étape 10 : barres = poids des segments ; heatmap = valeurs médianes (rouge = valeur élevée, y compris pour la récence).

11 Visualiser les segments avec une ACP

Comment : l'ACP projette les 3 variables sur 2 axes ; on colore chaque client selon son segment.

Code python

```
from sklearn.decomposition import PCA

pca = PCA(n_components=3, random_state=42).fit(X)
P = pca.transform(X)
print(pca.explained_variance_ratio_)
# [0.75, 0.19, 0.06]

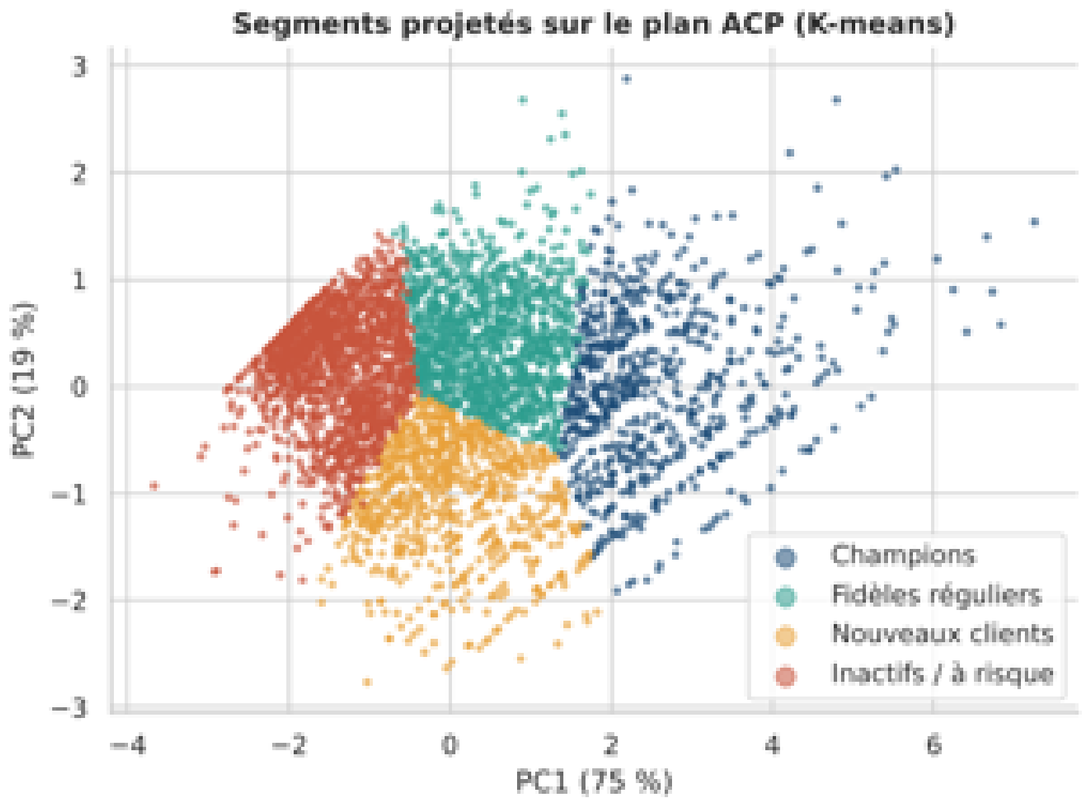
for i in sorted(NOMS):
    m = feat.Segment_id.values == i
    plt.scatter(P[m, 0], P[m, 1], s=7, alpha=.55,
                label=NOMS[i])
plt.xlabel("PC1"); plt.ylabel("PC2"); plt.legend()
```

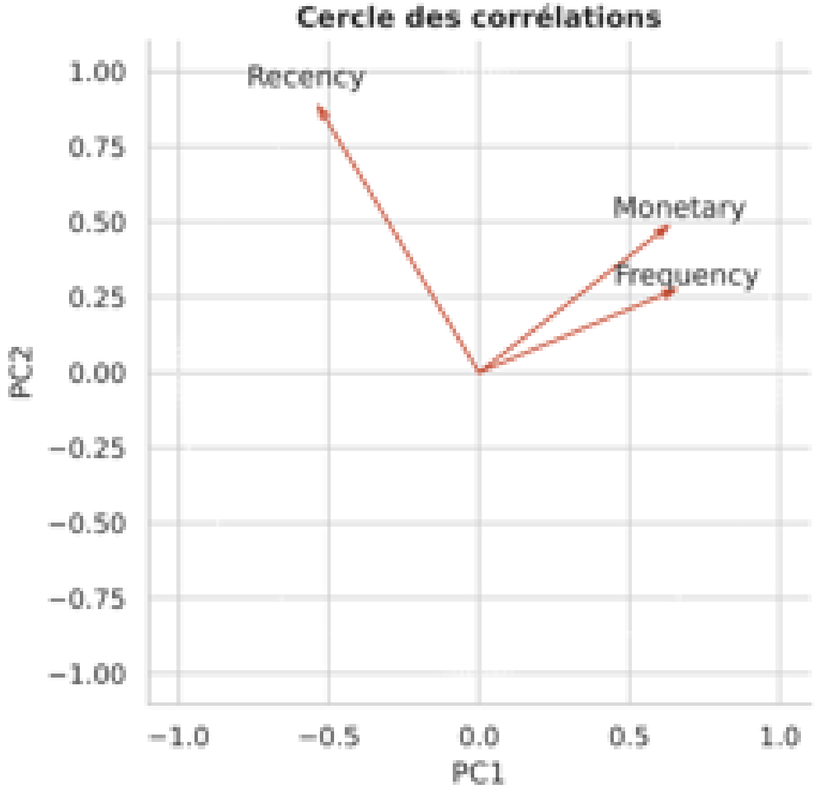
75 %
variance expliquée par PC1 (activité

19 %
variance expliquée par PC2 (récence)

Les segments forment un continuum : les frontières sont des seuils, pas des ruptures.

Le plan ACP confirme des segments ordonnés par l’intensité d’activité





Résultat du code de l’étape 11 : projection des 4 334 clients dans le plan (PC1, PC2) et cercle des corrélations.

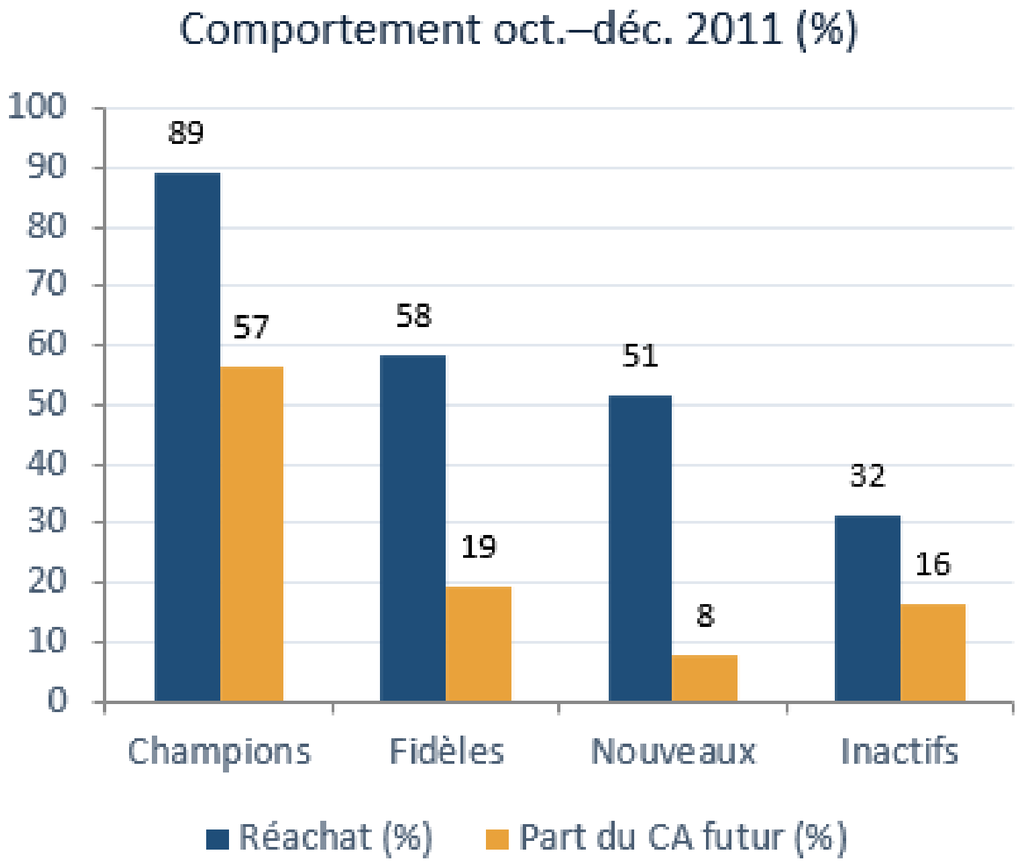
12 Valider dans le temps, sans fuite de données

Comment : segments construits avec le passé seul (avant le 01/10/2011), puis comparés au CA réellement observé après.

Code python

```
CUT = pd.Timestamp("2011-10-01")
v_tr = ventes[ventes.InvoiceDate < CUT]    # passé seul
a_tr = annul[annul.InvoiceDate < CUT]
f_tr = construire_features(v_tr, a_tr, CUT)
X_tr, _ = preparer(f_tr, core)
km_tr = KMeans(4, n_init=50,
               random_state=42).fit(X_tr)
f_tr["seg_id"] = reordonner(km_tr.labels_, f_tr)

futur = (ventes[ventes.InvoiceDate >= CUT]
         .groupby("CustomerID").Montant.sum())
f_tr["CA_futur"] = futur.reindex(f_tr.index).fillna(0)
f_tr["Rachat"] = (f_tr.CA_futur > 0).astype(int)
f_tr.groupby("seg_id").Rachat.mean()
```



Une action commerciale distincte par segment

1 Champions · 16 % clients, 64 % CA

Protéger : conseiller dédié, accès anticipé aux nouveautés, surveillance de la récence.

2 Fidèles réguliers · 27 % clients

Faire monter en gamme : ventes croisées, paliers de remise sur volume.

3 Nouveaux clients · 19 % clients

Obtenir le 2e achat : parcours d'accueil et relance dans les 30 jours.

4 Inactifs · 38 % clients, 6 % CA

Réactiver à coût plafonné : seuls 31 % rachètent, ne pas sur-investir.

Six limites à garder en tête avant toute décision

1 Clients anonymes

25 % des lignes sans identifiant : biais possible.

2 Période courte

12 mois, Noël et décembre 2011 tronqué.

3 Représentativité

Un seul commerçant, 83 % UK, clients B2B.

Six limites à garder en tête avant toute décision

4 Valeur ≠ CLV

Le CA passé ne garantit pas la valeur future.

5 Pas de vérité terrain

La qualité repose sur stabilité et cohérence métier

6 Usage pédagogique

Ne pas utiliser pour des décisions financières réelles

Ce qu’il faut retenir

64 %

du CA vient de 16 % des clients : les Champions

0,96

stabilité des segments K-means sur sous-échantillons

89 %

de réachat des Champions, contre 31 % pour les Inactifs